ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN D

GATEWAY / COLLECTOR & ĐỘ TIN CẬY MẠNG

Trạm trung chuyển dữ liệu, công cụ giả lập kiểm thử tải và kỹ thuật phân tích gói tin

---

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN D

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn phụ trách lớp trung chuyển giúp hệ thống phục vụ nhiều Agent: **Gateway** và **Collector**.

2. BẠN LÀM GÌ?

- Hiểu giao thức Protocol do Thành viên A thiết kế.
- Dựng môi trường Stub/Mock (FakeAgent và FakeCentral) để thử nghiệm độc lập.
- Nắm vững cơ chế hàng đợi có giới hạn để điều tiết lưu lượng gói tin.
- Xây dựng từng bước trạm trung chuyển dữ liệu Gateway và Collector.
- Kiểm tra kịch bản chuyển đổi dự phòng (Failover) khi máy chủ gặp sự cố.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo đo phần cứng máy trạm (việc của **Thành viên A**).
- Bạn **không cần** lo cấu trúc bảng CSDL SQL Server hay viết luật nghiệp vụ (việc của **Thành viên B**).
- Bạn **không cần** lo vẽ giao diện đồ họa WinForms (việc của **Thành viên C**).
- *Lưu ý:* Simulator và Wireshark là **công cụ hỗ trợ kiểm thử sau này**, không phải nghiệp vụ cốt lõi bạn cần lo ngay từ đầu.

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Đọc tài liệu để hiểu tại sao cần trạm Gateway khi phòng máy có nhiều máy tính, sau đó tìm hiểu cấu trúc đóng khung 4 byte độ dài của Protocol.

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `START_HERE.md`: Kiến trúc tổng quát và vai trò các trạm mạng.
2. `src/LabGuard.Protocol/FrameCodec.cs`: Cách cắt và ghép 4 byte độ dài khi trung chuyển dữ liệu.
3. `src/LabGuard.Protocol/WireEnvelope.cs`: Cấu trúc thông điệp chuẩn để kiểm tra tính toàn vẹn.
4. `src/LabGuard.Protocol/MessageTypes.cs`: Danh mục tên các loại thông điệp cần kiểm thử.

MỤC LỤC

1 Tổng quan vai trò Thành viên D **1**
1.1 Bài toán thực tế: Khi phòng máy có hàng trăm máy tính 1

2 Sáu chặng kỹ thuật của Thành viên D **2**
2.1 Chặng 1: VIỆC TIẾP THEO CỦA BẠN: Môi trường Stub độc lập (FakeAgent / FakeCentral) 2
2.2 Chặng 2: VIỆC TIẾP THEO: Hàng đợi thu gom an toàn (Bounded Channel) 3
2.3 Chặng 3: VIỆC TIẾP THEO: Chuyển tiếp hai chiều (Duplex Relay) & Tự phục hồi (Failover) 4
2.4 Công cụ hỗ trợ: Bộ giả lập phòng máy (Simulator) 5
2.5 Công cụ hỗ trợ: Bắt và phân tích gói tin bằng Wireshark 5
2.6 Chặng 6: VIỆC TIẾP THEO: Diễn tập kịch bản bảo vệ 5 phút ấn tượng 6

3 Năm bẫy lỗi lập trình Gateway & Kiểm thử mạng **7**

4 Bộ câu hỏi vấn đáp dành riêng cho Thành viên D (20 câu) **8**

1. Tổng quan vai trò của Thành viên D: Mở rộng quy mô & Độ tin cậy

TÓM TẮT 30 GIÂY

- Bạn phụ trách lớp trung chuyển giúp hệ thống phục vụ nhiều Agent: **Gateway** và **Collector**.
- Đóng vai trò lớp đệm mở rộng quy mô (Scale Layer): gom luồng, điều tiết lưu lượng và bảo đảm độ tin cậy.
- Nắm vững cơ chế hàng đợi có giới hạn để tiếp nhận gói tin mà không làm tràn bộ nhớ hay nghẽn máy chủ.
- Simulator và Wireshark là các công cụ hỗ trợ kiểm thử sau này để thẩm định năng lực chịu tải và độ ổn định.

1.1 Bài toán thực tế: Khi phòng máy có hàng trăm máy tính

Khi hệ thống LabGuard được nhân rộng cho cả một dãy phòng máy của trường đại học với hơn 500 đến 1,000 máy tính hoạt động cùng lúc:

- **Nguy cơ bão kết nối (Connection Storm):** Nếu hàng trăm máy cùng gửi số liệu trực tiếp vào máy chủ Central trên cổng 5000, máy chủ sẽ bị quá tải kết nối và nghẽn mạng.
- **Mạng giữa các phòng học (WAN / NAT):** Các máy trong từng phòng thường nằm sau router riêng, việc đặt một trạm thu gom (**Gateway**) tại mỗi phòng sẽ giúp tiết kiệm tối đa băng thông đường truyền.

KHÁI NIỆM MẠNG CỐT LÕI

Trọng trách cốt lõi của Thành viên D:

1. **Trạm thu gom Gateway (Collector):** Gom các bản tin số liệu từ các máy trong phòng, đưa vào hàng đợi an toàn và chuyển tiếp về máy chủ trung tâm.
2. **Môi trường giả lập độc lập (FakeAgent / FakeCentral):** Giúp nhóm kiểm thử sớm mà không phải đợi nhau viết xong code.
3. **Bộ giả lập phòng máy (Simulator):** Tạo 30–50 máy ảo gửi nhịp tim định kỳ 5s và tích hợp phím bấm bơm sự cố để trình diễn.
4. **Phân tích gói tin qua Wireshark:** Soi dòng byte nhị phân để chứng minh hệ thống đóng khung 4 byte Big-Endian chuẩn xác.

Hình này chỉ cần nhìn theo các khối chức năng từ trên xuống dưới theo thứ tự triển khai.



Figure 1: Lộ trình 6 giai đoạn kỹ thuật mở rộng của Thành viên D

Ý chính: Thành viên D đảm nhiệm tầng mở rộng (Gateway + Collector), bảo đảm hệ thống vận hành bền bỉ và chuyển tiếp thông suốt dưới tải cao.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Gateway là trạm trung gian kết nối giữa máy phòng học và máy chủ trung tâm.
- Simulator và Wireshark là công cụ hỗ trợ để thẩm định độ ổn định và kiểm chứng hệ thống dưới tải lớn.

2. Sáu chặng kỹ thuật của Thành viên D

TÓM TẮT 30 GIÂY

- Chặng 1: Xây dựng FakeAgent và FakeCentral để kiểm thử độc lập.
- Chặng 2–3: Lập trình hàng đợi Bounded Channel và chuyển tiếp hai chiều.
- Chặng 4–5: Viết bộ giả lập 50 máy ảo và bắt gói tin qua Wireshark.
- Chặng 6: Diễn tập kịch bản demo 5 phút trước hội đồng chấm thi.

2.1 Chặng 1: VIỆC TIẾP THEO CỦA BẠN: Môi trường Stub độc lập (FakeAgent / FakeCentral)

2.1.1 Tại sao cần môi trường Stub?

Khi làm việc nhóm 4 người, nếu bạn phải chờ bạn A viết xong Agent hoặc bạn B viết xong Central mới được thử code của mình thì tiến độ sẽ bị tắc nghẽn hoàn toàn. Các thành phần giả lập (**Stubs**) giúp bạn tự do thử nghiệm độc lập:

- **FakeAgent:** Một chương trình nhỏ giả vờ là máy sinh viên, tự động kết nối và gửi các gói tin mẫu chuẩn.
- **FakeCentral:** Một server nhỏ mở cổng 5000, lắng nghe kết nối và phản hồi các gói tin mẫu.

2.1.2 Khi chạy, chuyện gì xảy ra?

FakeAgent đọc trực tiếp 16 fixture JSON chuẩn trong `tests/LabGuard.Protocol.Tests/Fixtures/` rồi gửi qua mạng. Nhờ vậy, dữ liệu kiểm thử luôn khớp 100% với hợp đồng giao thức của nhóm.

2.1.3 Khi nào coi là xong? (PASS Criteria)

- Gửi và nhận chính xác 16 fixture mẫu mà không bị lỗi giải mã.
- Toàn bộ khung tin trao đổi đều có 4 byte Big-Endian đúng quy định.

2.2 Chặng 2: VIỆC TIẾP THEO: Hàng đợi thu gom an toàn (Bounded Channel)

2.2.1 Tại sao cần phần này?

Nếu Gateway nhận được gói tin nào từ máy sinh viên cũng lập tức bắn thẳng về Central thì đường truyền sẽ ngập tràn các gói tin nhỏ rời rạc. Ngược lại, nếu Gateway lưu tạm vào một hàng đợi không giới hạn trong bộ nhớ thì khi mạng WAN bị chậm hoặc đứt quãng, hàng triệu byte dữ liệu dồn về sẽ làm cạn kiệt RAM và gây sập chương trình.

2.2.2 Khi chạy, chuyện gì xảy ra?

Hàng đợi có giới hạn để bộ nhớ không tăng vô hạn. Khi hàng đợi đầy, producer phải chờ để hệ thống tự giảm tốc độ tiếp nhận. Cơ chế bảo vệ này được gọi là **điều tiết áp lực ngược (Backpressure)**.

Trong bản thiết kế của Gateway, hàng đợi thu gom được cấu hình dung lượng 100 phần tử (`BoundedChannelOptions(100)`) với chính sách chờ an toàn (`BoundedChannelFullMode.Wait`):

- Duy nhất một luồng nền lấy từng gói tin ra để gửi tuần tự về Central Server.
- Nếu hàng đợi đạt ngưỡng 100 phần tử, tác vụ đẩy tin tạm dừng bất đồng bộ cho tới khi hàng đợi có chỗ trống, ngăn ngừa hoàn toàn nguy cơ tràn bộ nhớ RAM.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Hạn mức 100 phần tử bảo đảm bộ đệm Gateway luôn ổn định, kiểm soát an toàn mức chiếm dụng RAM.
- Chính sách `FullMode.Wait` kích hoạt áp lực ngược, bảo vệ hệ thống không bị quá tải.

```
// Pseudocode minh hoa: Hang doi thu gom du lieu Bounded Channel
public class TelemetryCollector
{
    private readonly Channel<byte[]> _queue = Channel.CreateBounded<byte[]>(
```

```
        new BoundedChannelOptions(100) { FullMode = BoundedChannelFullMode.Wait
            });

    public async ValueTask EnqueueAsync(byte[] frame, CancellationToken ct)
        => await _queue.Writer.WriteAsync(frame, ct);

    public async Task ForwardAsync(Stream stream, CancellationToken ct)
    {
        await foreach (var frame in _queue.Reader.ReadAllAsync(ct))
            await FrameCodec.WriteFrameAsync(stream, frame, ct);
    }
}
```

Listing 1: Pseudocode minh họa: Hàng đợi thu gom Bounded Channel và Backpressure

2.2.3 Khi nào coi là xong? (PASS Criteria)

- Hàng đợi giữ tối đa 100 phần tử, producer tự động tạm dừng đồng bộ nhịp độ khi đầy.
- Bộ nhớ RAM luôn giữ ổn định, không xảy ra tràn bộ nhớ đệm.

2.3 Chặng 3: VIỆC TIẾP THEO: Chuyển tiếp hai chiều (Duplex Relay) & Tự phục hồi (Failover)

2.3.1 Tại sao cần chuyển tiếp hai chiều?

Dữ liệu trong phòng máy không chỉ đi một chiều từ máy trạm lên máy chủ. Khi máy chủ Central muốn phản hồi gói tin xác nhận (`HEARTBEAT_ACK`) hoặc gửi lệnh điều khiển (như khóa màn hình, khởi động lại), làm sao gửi đúng đến chiếc máy đích nếu mọi máy trạm đều kết nối tập trung qua Gateway?

Gateway phải đóng vai trò người phát thư: ghi nhớ máy trạm nào đang gắn với kết nối mạng nào để chuyển tiếp đúng địa chỉ. Kỹ thuật chuyển tiếp hai chiều này gọi là **Duplex Relay**.

2.3.2 Khi chạy, chuyện gì xảy ra?

Gateway lưu một cuốn sổ danh bạ các kết nối mạng đang hoạt động. Do nhiều luồng cùng đọc và ghi danh bạ này đồng thời, ta sử dụng bảng tra cứu an toàn đa luồng (`ConcurrentDictionary<string, TcpClient> _agentSockets`). Khi nhận bản tin điều khiển từ Central gửi về, Gateway mở phong bì, đọc mã máy `deviceId`, tra cứu kết nối socket tương ứng trong danh bạ và chuyển tiếp ngay tới đúng máy trạm đó.

2.3.3 Kịch bản tự phục hồi khi đứt mạng (Failover)

Nếu đường truyền mạng giữa Gateway và Central bị lỏng dây hoặc chập chờn:

1. Gateway không hoảng loạn làm sập kết nối với các máy trạm trong phòng học. Gateway tiếp tục gom số liệu vào hàng đợi an toàn.
2. Gateway tự động kích hoạt cơ chế **tự phục hồi (Failover)** bằng cách lùi dần thời gian chờ kết nối lại (**Exponential Backoff**): lần đầu chờ 1 giây, lần hai 2 giây, rồi 4 giây, 8 giây... để tránh làm nghẽn đường truyền.
3. Ngay khi mạng thông suốt trở lại, Gateway chuyển tiếp bù toàn bộ các gói tin đang chờ trong hàng đợi về Central một cách trơn tru.

2.4 Công cụ hỗ trợ: Bộ giả lập phòng máy (Simulator)

Chặng 4 — Mã tác vụ nội bộ: LG-D-004

2.4.1 Tại sao cần phần này?

Trong buổi thi kết thúc môn, bạn không thể mang 50 máy tính thật lên bàn thi. Bộ giả lập `LabGuard.Simulator` giúp bạn tạo ra 30–50 máy ảo ngay trên chiếc laptop của mình để chứng minh hệ thống hoạt động mượt mà dưới tải thực tế.

2.4.2 Khi chạy, chuyện gì xảy ra?

Simulator khởi chạy 30–50 tác vụ `Task.Run` song song. Mỗi tác vụ đóng vai một máy sinh viên gửi nhịp tim mỗi 5 giây. Menu bàn phím cho phép bạn chủ động tạo sự cố:

- Phím [1]: Bơm tải CPU 95% cho máy ảo 05 → Central lập tức báo động đỏ.
- Phím [2]: Hạ CPU máy ảo 05 về 20% → Central tự động đóng sự cố.
- Phím [3]: Rút mạng máy ảo 12 → Sau hơn 20 giây (> 20s), Central báo `OFFLINE`.
- Phím [4]: Cắm lại mạng cho máy ảo 12 → Kết nối lại bình thường.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Mỗi máy ảo chạy độc lập trong vòng lặp `PeriodicTimer(5s)`.
- Biến cờ `IsNetworkCut` giúp mô phỏng tình huống rút dây mạng nhanh chóng.

```
// Pseudocode minh hoa: May ao VirtualAgent gui nhip tim va so lieu
while (await timer.WaitForNextTickAsync(ct))
{
    if (IsNetworkCut) continue; // Gia lap rut day mang
    await SendHeartbeatAsync(stream, ct);
    await SendMetricBatchAsync(stream, SimulatedCpu, ct);
}
```

Listing 2: Pseudocode minh họa: Vòng lặp mô phỏng VirtualAgent

2.4.3 Khi nào coi là xong? (PASS Criteria)

- Tạo 30–50 máy ảo đồng thời mà không làm đơ laptop.
- Menu phím bấm bơm lỗi phản hồi tức thời trên màn hình Admin.

2.5 Công cụ hỗ trợ: Bắt và phân tích gói tin bằng Wireshark

Chặng 5 — Mã tác vụ nội bộ: LG-D-005

2.5.1 Tại sao cần phần này?

Wireshark hỗ trợ kiểm chứng dữ liệu thực tế trên đường truyền, giúp bạn quan sát trực tiếp luồng byte nhị phân:

1. Mở Wireshark, chọn card mạng `Adapter for loopback` (nếu chạy nội bộ).
2. Gõ bộ lọc hiển thị: `tcp.port == 5000`.

3. Bấm vào một gói tin và chỉ cho thầy cô thấy:
   - Bắt tay 3 bước TCP ([SYN], [SYN, ACK], [ACK]).
   - 4 byte Big-Endian ở đầu khung tin (ví dụ: `00 00 00 9C` là 156 bytes).
   - Dữ liệu JSON của `WireEnvelope` với đúng 6 trường quy chuẩn.

2.5.2 Bảng soi từng byte khung tin trên Wireshark

Vị trí Byte	Giá trị Hex / Ký tự	Giải thích kỹ thuật
`0x0000--0x0003`	`00 00 00 9C`	4 byte độ dài Big-Endian = 156 byte payload theo sau.
`0x0004--0x0011`	`{"version":1,`	Phiên bản giao thức `version = 1`.
`0x0012--0x002B`	`"messageType":"HEARTBEAT",`	Tên thông điệp chuẩn trong `MessageTypes.cs`.
`0x002C--0x0058`	`"requestId":"d2a7...-90ef",`	Mã UUID v4 duy nhất 36 ký tự.
`0x0059--0x0078`	`"deviceId":"AGENT-LAB01-05",`	Tên thiết bị gửi bản tin.
`0x0079--0x00A8`	`"sentAtUtc":"2026-03-29...",`	Dấu thời gian UTC chuẩn có đuôi `Z`.
`0x00A9--0x009F`	`"payload":{...}}`	Nội dung nghiệp vụ chi tiết của thông điệp.

2.6 Chặng 6: VIỆC TIẾP THEO: Diễn tập kịch bản bảo vệ 5 phút ấn tượng

Mã tác vụ nội bộ: LG-D-006

Table 1: Kịch bản trình diễn nghiệm thu 5 phút thuyết phục trước hội đồng thi

Thời gian	Người thực hiện	Hành động trên màn hình	Ý nghĩa kỹ thuật
Phút 0–1	Bạn B & Bạn C	Khởi động Central Server trên cổng 5000 và mở Admin Dashboard.	Máy chủ đã mở cổng TCP sẵn sàng tiếp nhận.
Phút 1–2	Bạn D	Chạy `LabGuard.Simulator` kích hoạt 30 máy trạm ảo.	Màn hình Dashboard lập tức sáng 30 dòng xanh lá rực rỡ.
Phút 2–3	Bạn D	Bấm phím [1] bơm CPU 95% lên máy ảo 05.	Sau 3 chu kỳ đo liên tiếp, Dashboard báo động đỏ `HIGH_CPU`.
Phút 3–4	Bạn D	Bấm phím [3] ngắt mạng máy ảo 12.	Sau hơn 20 giây (> 20s), hệ thống báo máy `OFFLINE`.
Phút 4–5	Bạn D	Mở Wireshark soi gói tin với bộ lọc `tcp.port == 5000`.	Chỉ rõ tiêu đề 4 byte Big-Endian và phong bì 6 trường chuẩn.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Dùng Bounded Channel để bảo vệ bộ nhớ RAM không bị tràn khi mạng chậm.
- Nắm vững thao tác mở Wireshark lọc cổng 5000 để kiểm chứng dữ liệu thực tế trên đường truyền.

3. Năm bẫy lỗi lập trình Gateway & Kiểm thử mạng

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Tưởng lầm một lệnh Write trên TCP luôn tạo ra một gói tin mạng
Giao thức TCP là một dòng byte liên tục, không có ranh giới tin nhắn cố định. Nếu Gateway đọc từ Agent rồi ghi thẳng sang Central mà không dùng `FrameCodec`, hai gói tin nhỏ liên tiếp có thể bị dính chùm hoặc bị cắt vụn.
Cách làm đúng: Luôn giải mã nguyên vẹn từng khung tin qua `FrameCodec.ReadFrameAsync()` trước khi chuyển tiếp.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Chuyển tiếp mảng byte thô sẽ làm vỡ gói tin ở đầu nhận.
- Đọc trọn vẹn 1 khung tin nhị phân rồi mới chuyển tiếp sang luồng khác.

```
// [SAI] Chuyen tiep byte tho gay vo khung tin o dich:
int read = await agentStream.ReadAsync(buffer, ct);
await centralStream.WriteAsync(buffer.AsMemory(0, read), ct);

// [DUNG] Doc tron ven 1 khung tin nhip phan roi moi chuyen tiep:
byte[]? frame = await FrameCodec.ReadFrameAsync(agentStream, ct);
if (frame != null) await FrameCodec.WriteFrameAsync(centralStream, frame, ct);
```

Listing 3: So sánh Bẫy 1: Chuyển tiếp luồng thô vs Chuyển tiếp theo khung tin

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Nghẽn đầu hàng (Head-of-Line Blocking) do dùng hàng đợi không giới hạn
Nếu sử dụng `Queue<byte[]>` không giới hạn dung lượng, khi mạng chậm gói tin sẽ dồn ứ vô tận trong RAM dẫn đến tràn bộ nhớ (*OutOfMemoryException*).
Cách làm đúng: Luôn sử dụng `Channel.CreateBounded` với giới hạn dung lượng an toàn để tránh tăng RAM vô hạn.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Tranh chấp truy cập bảng định tuyến phiên
Khi 50 máy con kết nối và ngắt kết nối đồng thời, việc tra cứu socket đích sẽ gây lỗi đa luồng nếu dùng `Dictionary` thông thường.
Cách làm đúng: Luôn dùng `ConcurrentDictionary<string, TcpClient>`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Tạo quá nhiều Thread độc lập trong bộ giả lập Simulator
Nếu tạo 50 máy ảo bằng cách gõ 50 lần `new Thread()`, máy tính sẽ bị ngốn hàng chục MB bộ nhớ và làm CPU giật lag do chuyển ngữ cảnh liên tục.
Cách làm đúng: Tận dụng `Task.Run` kết hợp `async/await` và `PeriodicTimer`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Quên giải phóng kết nối khi máy trạm ngắt đột ngột
Khi máy trạm bị tắt nguồn hoặc rút cáp, nếu vòng lặp không bắt lỗi đóng luồng thì socket sẽ bị treo vĩnh viễn, làm rò rỉ tài nguyên mạng của hệ thống.
Cách làm đúng: Luôn đặt luồng socket trong khối lệnh `using`.

4. Bộ câu hỏi vấn đáp dành riêng cho Thành viên D (20 câu)

1. **Tại sao phải dùng Wireshark để soi gói tin thay vì chỉ đọc log trên console?**
   Trả lời: Wireshark hỗ trợ kiểm chứng dữ liệu thực tế trên đường truyền ở tầng truyền vận (TCP/IP), chứng minh hệ thống tuân thủ chặt chẽ tiêu chuẩn đóng khung 4 byte Big-Endian và mô hình kết nối tin cậy.

2. **Bộ lọc nào trên Wireshark cho phép cô lập chính xác dữ liệu của LabGuard?**
   Trả lời: Dùng biểu thức lọc hiển thị `tcp.port == 5000` để thu thập toàn bộ các gói tin TCP trao đổi giữa Agent, Gateway và Central Server.

3. **Tại sao đồ án cần xây dựng các bản Stub (FakeAgent, FakeCentral)?**
   Trả lời: Nhằm phá vỡ sự phụ thuộc chéo về tiến độ giữa các thành viên, cho phép kiểm thử độc lập chức năng chuyển tiếp của Gateway ngay từ những ngày đầu.

4. **Bộ giả lập (Simulator) có vai trò gì trong buổi nghiệm thu đồ án?**
   Trả lời: Giúp tái hiện môi trường thực tế với 20–50 máy trạm hoạt động đồng thời, đóng vai trò công cụ hỗ trợ kiểm chứng năng lực chịu tải và thuật toán xử lý sự cố đa luồng của máy chủ.

5. **Tại sao nhịp tim gửi 5 giây một lần mà lại chờ hơn 20 giây mới báo Offline?**
   Trả lời: Nhằm tránh báo động giả do trễ mạng hoặc suy giảm kết nối nhất thời. Máy chủ quy định ngưỡng quá hạn gấp 4 lần chu kỳ gửi ($4 \times 5\text{s} = 20\text{s}$).

6. **Làm thế nào để Gateway tránh việc làm nghẽn mạng WAN?**
   Trả lời: Bằng kỹ thuật gom cụm (Batching) nhiều bản tin đo đạc nhỏ từ các Agent trong phòng máy vào hàng đợi Bounded Channel trước khi chuyển tiếp thành từng khối lớn về Central Server.

7. **Tại sao hàng đợi thu gom của Gateway phải là hàng đợi có giới hạn (Bounded)?**
   Trả lời: Để ngăn ngừa nguy cơ cạn kiệt bộ nhớ RAM khi đường truyền mạng WAN bị gián đoạn hoặc quá tải. Hàng đợi giới hạn 100 phần tử kích hoạt cơ chế điều tiết áp lực ngược (Backpressure) với chính sách `FullMode.Wait`, buộc bên gửi tạm dừng bất đồng bộ để hệ thống tự điều hòa nhịp độ an toàn.

8. **Cơ chế Exponential Backoff trong quá trình kết nối lại hoạt động ra sao?**
   Trả lời: Sau mỗi lần kết nối lại thất bại, thời gian chờ tăng gấp đôi (1s, 2s, 4s, 8s... trần 30s), tránh gây ra hiện tượng bão kết nối làm tê liệt máy chủ.

9. **Làm thế nào để Simulator tạo ra 50 máy ảo mà không làm treo ứng dụng?**
   Trả lời: Dùng `Task.Run` tận dụng ThreadPool và cơ chế I/O Completion Ports (IOCP) của hệ điều hành, cho phép tạo 50–100 máy ảo chạy trên chỉ vài luồng hệ thống mà không tốn RAM.

10. **Tiêu đề 4 byte Big-Endian hiển thị ra sao trong cửa sổ Byte Viewer của Wireshark?**
    Trả lời: Hiển thị dưới dạng 4 byte hex liên tiếp ở đầu payload TCP, ví dụ: `00 00 00 A4` nghĩa là khung tin dài 164 byte.

11. **Tại sao phải chờ hơn 20 giây thì Central Server mới báo máy trạm bị ngắt mạng?**
Trả lời: Vì chu kỳ nhịp tim là 5 giây. Máy chủ cho phép trễ tối đa 4 chu kỳ liên tiếp (tổng cộng > 20 giây) để tránh báo động giả khi mạng phòng máy bị nghẽn tức thời.

12. **Kỹ thuật Bơm lỗi (Fault Injection) mang lại lợi ích gì cho đồ án?**
Trả lời: Cho phép chủ động kiểm chứng độ tin cậy và khả năng tự phục hồi của toàn bộ hệ thống ngay tại bàn thi mà không cần can thiệp vào thiết bị phần cứng thật.

13. **Làm thế nào để đo độ trễ mạng (RTT) của gói tin nhịp tim?**
Trả lời: Tính khoảng thời gian từ khi Client gửi bản tin `HEARTBEAT` đến khi nhận được bản tin `HEARTBEAT_ACK` tương ứng từ Central Server.

14. **Giới hạn khung tin 256 KiB có ảnh hưởng gì tới tầng Gateway không?**
Trả lời: Không ảnh hưởng vì Gateway kiểm tra và duy trì nguyên vẹn kích thước khung tin của giao thức chuẩn `FrameCodec.MaxPayloadSize`.

15. **Tại sao khi thử tải lại không chạy trên 50 máy tính thật?**
Trả lời: Vì sinh viên không thể có 50 máy tính thật để thử nghiệm hàng ngày. Bộ giả lập chứng minh năng lực thiết kế phần mềm có thể đáp ứng quy mô lớn ngay trên môi trường phát triển.

16. **Hiện tượng SYN Flood là gì và ảnh hưởng gì tới máy chủ?**
Trả lời: Kẻ tấn công gửi dồn dập các gói tin `SYN` giả mạo mà không gửi lại gói `ACK` thứ ba, làm cạn kiệt hàng đợi kết nối đang chờ (*Backlog Queue*) của máy chủ.

17. **Cơ chế Backlog trong lệnh Listen(int backlog) có ý nghĩa gì?**
Trả lời: Là độ dài tối đa của hàng đợi chứa các kết nối đã hoàn thành bắt tay TCP 3 bước nhưng ứng dụng chưa kịp gọi hàm `Accept`.

18. **Làm sao để đảm bảo các máy trạm ảo trong Simulator không gửi trùng mã RequestId?**
Trả lời: Mỗi thông điệp đều sinh mã `Guid.NewGuid().ToString("D")` độc lập, bảo đảm tính duy nhất tuyệt đối trên toàn hệ thống.

19. **Khi tắt đột ngột chương trình Simulator, chuyện gì xảy ra trên máy chủ?**
Trả lời: Hệ điều hành Windows sẽ tự động gửi gói tin cờ `RST` hoặc `FIN` tới máy chủ, máy chủ lập tức giải phóng toàn bộ tài nguyên socket của các máy ảo đó.

20. **Thành công lớn nhất trong vai trò của Thành viên D là gì?**
Trả lời: Là làm chủ được năng lực mở rộng hệ thống theo chiều ngang, tư duy kiến trúc phân tầng mạng và khả năng dùng công cụ chuẩn quốc tế (Wireshark) để bảo vệ giải pháp trước hội đồng thi.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN D —